

# Instrucciones para el Candidato

---

## 📋 Objetivo

Demostrar tus habilidades como desarrollador(a) backend/frontend en Laravel identificando y corrigiendo los problemas que existen en este sistema. La prueba está diseñada para durar **máximo 1 día** (6-8 horas efectivas).

**Nota importante:** Este sistema utiliza **relaciones simbólicas** en la base de datos (no foreign keys físicas). Las relaciones se manejan por convención de nombres ( `*_id` ) a nivel de aplicación. No se utiliza `SoftDeletes` ni `FormRequest` .

---

## 📋 Entorno sugerido

- **PHP:** 8.2 o superior
  - **Composer:** 2.x
  - **Base de datos:** SQLite (ya configurada) o MySQL si prefieres
  - **Node.js:** 18+ (si decides agregar assets con Vite)
  - **Editor:** VS Code, PHPStorm, etc.
- 

## 📋 Pasos iniciales

1. **Instalar y levantar** el proyecto siguiendo `README.md` .
  2. **Explorar** las rutas: Dashboard, Alumnos, Programas, Pagos.
  3. **Revisar** el código de los controladores, modelos, migraciones y vistas.
  4. **Identificar** qué está mal (funcionalmente, de seguridad, de performance, de arquitectura).
  5. **Corregir** los problemas que consideres más críticos.
  6. **Documentar** tus decisiones en el archivo `RESPUESTA.md` (créalo en la raíz).
- 

## 📋 Qué documentar en tu `RESPUESTA.md`

Tu respuesta debe responder al menos estas preguntas para cada corrección importante:

1. **¿Qué problema encontraste?** (describe el bug o la mala práctica)
  2. **¿Por qué es un problema?** (impacto en seguridad, performance, mantenibilidad)
  3. **¿Cómo lo solucionaste?** (código o enfoque)
  4. **¿Qué herramientas o patrones usaste?** (ej: Eloquent eager loading, Query Builder, validaciones inline, Database Transactions, etc.)
  5. **¿Cómo lo verificaste?** (test, log, query log, debugbar, etc.)
  6. **¿Qué optimizaste?** (índices, paginación, caché, chunk, etc.)
- 

## 📋 Áreas de enfoque (no tienes que corregir todo)

Prioriza las que más se ajusten al perfil buscado. Aquí hay algunas áreas con ejemplos de problemas:

## A. Base de datos y modelos

- ¿Por qué hay campos `text` para montos, matrículas y fechas?
- ¿Faltan índices en las tablas?
- ¿Por qué `tags` no está casteado a `array` ?
- **NO evaluar:** `foreign keys` (las relaciones son simbólicas), `SoftDeletes` (no se usa en este sistema).

## B. Seguridad

- ¿Hay posibilidad de SQL injection en algún controlador?
- ¿El middleware de roles usa comparación estricta?
- ¿Las rutas de eliminación usan el verbo HTTP correcto?
- ¿Los endpoints API están protegidos?

## C. Performance

- ¿El listado de alumnos pagina o carga todo?
- ¿Hay N+1 queries en las vistas?
- ¿El dashboard hace cálculos en PHP que podrían hacerse en SQL?
- ¿Los selects de alumnos/sedes cargan miles de registros?

## D. Lógica de negocio y validaciones

- ¿Se puede registrar un pago negativo?
- ¿Se valida que un alumno exista antes de registrar su pago?
- ¿La creación de alumno + inscripción + pago es atómica?
- ¿Se pueden eliminar alumnos con pagos pendientes?

## E. Frontend y UX

- ¿El select de alumnos en pagos es usable con 1000+ registros?
- ¿Falta algún botón de confirmación antes de eliminar?
- **NO evaluar:** `old()` en formularios (no es parte del flujo de este sistema).

## F. Arquitectura y buenas prácticas

- ¿Los jobs manejan excepciones?
- ¿El exporte de CSV usa `chunk` para no saturar memoria?
- ¿Hay transacciones en operaciones multi-tabla?
- ¿Se usa caché en queries repetidas del dashboard?
- **NO evaluar:** `FormRequest` (no se usa en este sistema; validar inline o en helpers).

---

## 📄 Entregables mínimos

1. **Crea un repositorio propio** (GitHub, GitLab, Bitbucket, etc.) con tu código corregido y tu `RESPUESTA.md`.
  2. **Archivo `RESPUESTA.md`** en la raíz del proyecto, explicando tus decisiones.
  3. (Opcional) Tests de PHPUnit que verifiquen al menos 2 correcciones críticas.
-

## 📁 Envío

Al finalizar, envía el **enlace a tu repositorio** y tu `RESPUESTA.md` al correo: `ltepepa@unisant.edu.mx`

**Nota:** Asegúrate de que el repositorio sea público o que nos compartas acceso de lectura.

---

## 📁 Reglas

- No puedes reescribir el proyecto desde cero. Debes trabajar sobre la base existente.
  - Puedes agregar packages de Composer si los justificas (ej: `laravel-debugbar` , `spatie/laravel-permission` ).
  - No se espera que todo quede perfecto; prioriza lo que más valor aporte.
  - **NO agregar foreign keys físicas** en la base de datos (las relaciones son simbólicas).
  - **NO agregar SoftDeletes** (no es parte del patrón de este sistema).
  - **NO usar FormRequest** (valida inline o en helpers).
- 

## 📁 Consejos

- **Lee primero, codea después.** Dedicar los primeros 30 min a entender el código.
  - Usa `php artisan db:show` o `php artisan migrate:status` para ver migraciones.
  - **Activa el query log** ( `DB::enableQueryLog()` ) para detectar N+1.
  - **No te atores en una sola cosa.** Si algo te bloquea, anótalo y sigue.
  - **Documenta mientras avanzas.** Es más fácil que hacerlo al final.
- 

## 📁 Dudas

Si tienes alguna duda sobre el alcance de la prueba, escríbela en tu `RESPUESTA.md` y explica cómo la interpretaste.

¡Éxito! 📁